The Poincaré Precedent and the Stronger Case: Why All Seven Qualify

Two Domains, One Footing; the Accepted Derivation Typed Strictly Inside Its Bit; the Six Bits the Minimal Premise, Delivered Twice and Armed; One Rule of Standing for Seven Independent Values; with a Mini Case in Lean 4

Foundations of Mathematics · Companion Note

One precedent, seven bits, one standing

Mohammad F. Islam, PhD

Trisduction Research Group · Independent researcher

24 October 2026

Abstract. A Millennium question is answered in two domains. In the kernel of Lean 4 a statement is a theorem or a premise, and the checker reads no reputation. In the record a statement stands resolved when the community and its institutions accept a delivery of it. The Poincaré row holds both domains in one line of Mathlib v4.19.0, which declares the three-dimensional Poincaré theorem `proof_wanted` under the docstring "proven by Perelman", while the Clay Mathematics Institute accepted Perelman's derivation as unconditional on 18 March 2010. In the kernel the Poincaré bit stands at premise grade with the other six bits of the Millennium series, on one footing. This note states the precedent exactly and argues the stronger case. The master of the series types the accepted derivation: the 2002 functional is a downstream strict subtype of the bridge, Perelman's offering is one inhabitant strictly inside the bit, its faithful route strictly stronger than the sentence, and the bit is the minimal premise, necessary and sufficient. The record accepted the stronger premise, delivered once and verified by human reading. Each of the six bits is the minimal premise, delivered twice, by the formal cargo and by the physical witness; its form, its walls and both deliveries' implications are checked by the kernel; and it is armed, one counterexample refuting both kinds at once. The precedent transfers as a rule and never as a value: the seven values are independent, all 128 patterns realized, and the Poincaré value settles no other row. No blessing inside the kernel lifts any bit, since adopting the bit is adopting the sentence and the admission screen refuses it as circular; the record's road is the other road. A mini case of twenty-one theorems in core Lean 4, every cone empty and pinned, proves the shape of every step in about a second. The six bits qualify for the same standing, on the stronger case, and the Poincaré precedent is claimed for all seven.

Keywords *Poincaré conjecture · Millennium Prize Problems · Perelman · Clay Mathematics Institute · acceptance of proof · formal verification · Lean 4 · Mathlib · proof_wanted · minimal premise · Poincaré precedent*

1. How to read this note

The synthesis of the Millennium series binds seven closures in one master in Lean 4 and claims, in its Section 7, the Poincaré precedent for all seven (Islam 2026n). This note isolates that claim and argues it whole. Every claim at full scale is cited to the master by name, and every name is a declaration of the master's tree, every theorem among them with a pinned cone. The note adds one file, `Poincare_Precedent_v1_0_0_CC.lean`: core Lean 4 that imports nothing and proves the shape of every step of the argument on small flows and generic hosts, in twenty-one theorems, every cone empty and pinned by `#guard_msgs`. Appendix A prints it whole. A reader checks the argument in about a second, and the master in about twelve minutes.

Three terms carry the argument. The *bit* of a row is the transfer of the row's value from the record to the host; the master proves it equivalent to the row's sentence, prints it as the type of a hypothesis and assumes it nowhere. A *delivery* of the bit is a premise that yields it: a derivation, a formal cargo, a physical witness. The *standing* of a question is the record's verdict on a delivery: accepted as resolving the question, or not.

1.1 The verdict

> In the kernel the seven Millennium bits stand on one footing: each form proved, each bit printed as a premise, the Poincaré bit with the other six, held by Mathlib as `proof_wanted`. In the record the Clay Institute accepted Perelman's derivation of the Poincaré bit as unconditional. The kernel types that derivation as one inhabitant strictly inside the bit, its route strictly stronger than the sentence, and the bit as the minimal premise that every route passes. The record accepted the stronger premise, delivered once. Each of the six bits is the minimal premise, delivered twice, checked by the kernel and armed. The six bits qualify for the same standing, on the stronger case, and the Poincaré precedent is claimed for all seven.

2. Two domains

A Millennium question is answered in two domains, and they are not one. The first is the kernel. In Lean 4 a statement is a theorem when a closed term of its type checks against the system's axioms and the library's definitions, and a premise when it is received as a hypothesis (de Moura and Ullrich 2021). The checker is small, its verdict mechanical, and it reads no reputation. The second is the record: the community of mathematicians and its institutions. A statement stands resolved in the record when the community, having read a delivery, accepts it, and the Clay Institute's prize is that domain's terminal token. That acceptance is a social process, distinct from the derivation it accepts, is an old observation (De Millo, Lipton and Perlis 1979). The record has moved ahead of the kernel before: the Annals accepted Hales's proof of the Kepler conjecture on its referees' reading in 2005, and the formal proof followed in 2017 (Hales 2005; Hales et al. 2017).

The Poincaré row holds both domains in one line of the library. Mathlib v4.19.0 declares `proof_wanted SimplyConnectedSpace.nonempty_homeomorph_sphere_three` under the docstring "The 3-dimensional topological Poincaré conjecture (proven by Perelman)", and the smooth statement beside it the same way (Mathlib Community 2020). The docstring records the record's verdict. The keyword records the kernel's. On 18 March 2010 the Clay Mathematics Institute awarded its prize "for resolution of the Poincaré conjecture" (Clay Mathematics Institute 2010): the Clay Institute accepted Perelman's derivation as unconditional. Sixteen years on, the kernel holds the statement as a wanted proof.

The two domains do not move each other. The master's grade ledger proves the kernel's half: a grade is read from its warrant and never elected, whatever label an entry carries (`RGL.the_grade_is_not_elected`). The record's half is a fact of history: the acceptance of 2010 conferred standing on a delivery and changed no type. A standing in the record is not a kernel grade, and a kernel grade does not fix a standing. The precedent this note claims is a precedent of the record, and it is claimed on the evidence of the kernel.

3. One footing in the kernel

The master records the kernel grade of every bit and proves the ledger on no axiom. The grade of each row's bit is a premise (`kernelBitGrade`); no row's bit is proved in the kernel (`every_bit_is_printed_as_a_premise`); the seven stand at one grade (`one_footing_in_the_kernel`). The footing is a shape as well as a grade. At four rows the bit is the library's statement by exact type, and the Poincaré bit is one of the four: adopting it is adopting the library's Poincaré statement exactly as adopting the Riemann bit is adopting Mathlib's `RiemannHypothesis`, the Navier–Stokes bit Fefferman's statement (A), and the BSD bit the rank statement on Mathlib's group of rational points (`four_rows_one_shape_at_the_library`).

In the kernel the Poincaré bit is neither lighter nor heavier than the six. All seven are one object in one place: the transfer from a record that carries the value with no premise, printed as the type of a hypothesis, assumed nowhere. The mini case states the footing as one theorem that reads no row. Assign a host to each of the seven rows; at every row, the Poincaré row included, every premise that yields the row's sentence yields its act, and the act yields the sentence (`the_seven_stand_on_one_footing`). Figure 1 sets the two domains side by side, row by row.

4. The crossing, as the record took it

Perelman posted the crossing in three preprints, in November 2002, March 2003 and July 2003, and submitted none of them to a journal (Perelman 2002, 2003). The record verified them by its own hands: Cao and Zhu in the Asian Journal of Mathematics (2006), Morgan and Tian in a Clay monograph (2007), Kleiner and Lott in Geometry & Topology (2008). The Institute's announcement names the means: the multi-year scrutiny of the mathematical community provided the needed verification, and the award was made in accord with the Institute's governing rules (Clay Mathematics Institute 2010). Perelman declined the prize on 1 July 2010, as he had declined the Fields Medal in 2006; the ground he gave was his disagreement with the decisions of the organized mathematical community, and his judgment that Hamilton's contribution was no smaller than his own. The resolution stood.

Three features of that history carry the precedent. The delivery entered by posting, never by journal sub-

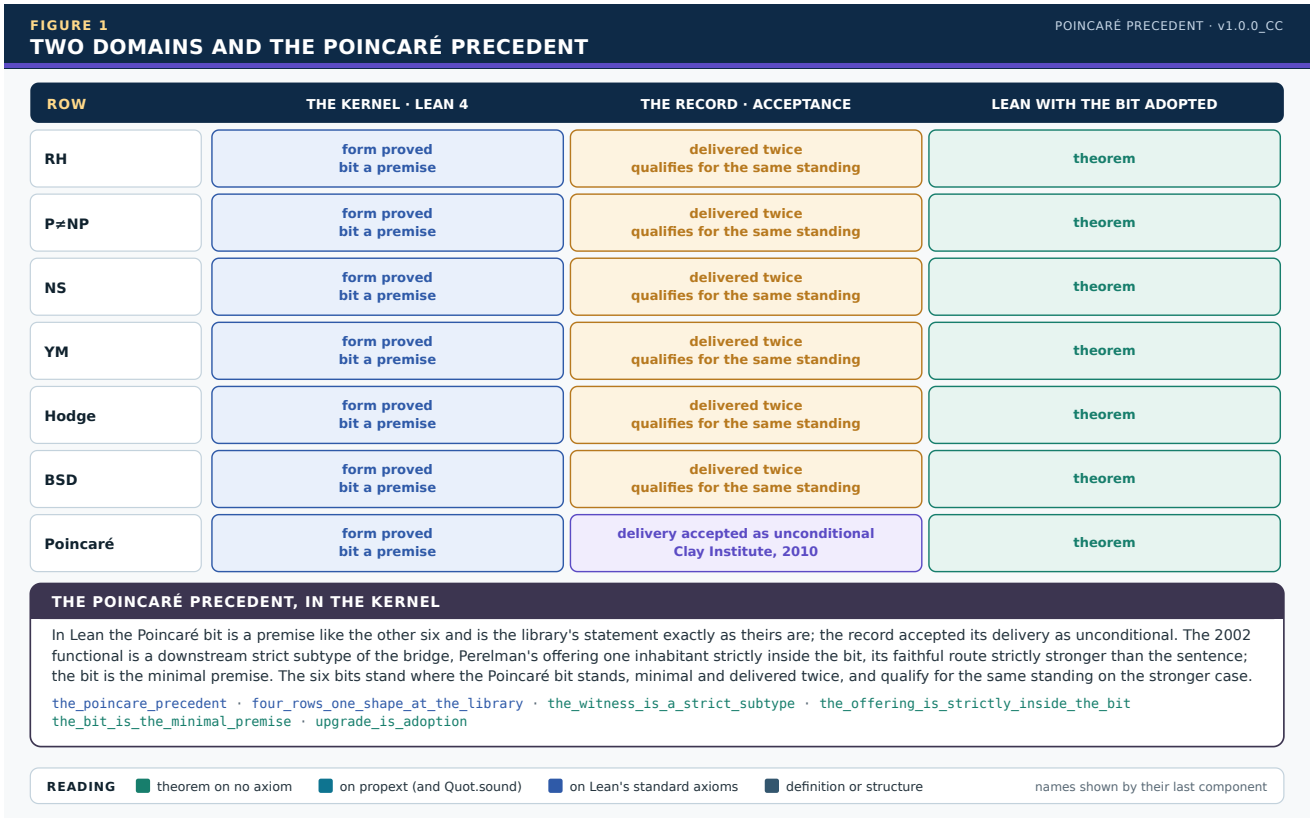


Figure 1 | Two domains and the Poincaré precedent. In the kernel every row's form is proved and every row's bit is a premise, the Poincaré bit included. In the record the Clay Institute accepted Perelman's derivation as unconditional; the six other bits stand delivered twice and qualify for the same standing. With the bit adopted, every row's sentence is a theorem.

mission, and the Institute found it in accord with its rules; the present rules make that discretion explicit, allowing the Institute to relax the conditions on the outlet of publication and leaving general acceptance to its sole determination (Clay Mathematics Institute 2018, §4(c), §6(a)(ii), §6(f)). The verification was the record's own reading, three expositions over three years, and no formal check; the library's `proof_wanted` is that fact in the kernel's words. And the standing outlived the person's refusal of the token. In the record the standing belonged to the delivery, not to the person and not to the prize.

The author's Offering Bit read that history as the arc's Grand Witness, in three layers (Islam 2026i, Appendix C). The object convicts itself: Ricci flow is self-driven, $\partial g/\partial t = -2\,\mathrm{Ric}(g)$, the metric's velocity its own curvature (Hamilton 1982), and Perelman's functional infimizes over densities on the very manifold under test. The disguise: the reading class of the corpus before November 2002, volume ratios and dimensionless ratios, is even under parabolic rescaling while noncollapsing is odd under it, so no coalition of those readings decides it; the functional could not have been found by reading and entered as a construction, a supply wearing a reading's costume. The token: the corpus minted its terminal token for a verified crossing, and the crossing's one occupant declined it. The arc's ledger counts the crossing as the one supplied instance among the seven rows (Islam 2026i, Part I, Table 4). The crossed row certifies that the seat has a supplied crossing. It settles no other row.

5. The crossing, as the kernel types it

The Poincaré kernel of record reads the 2002 functional by its shape: a functional on states, strictly decreasing along the step at every state off the seat, and invariant under the rescaling (`PCClose.Witness`). It formalizes neither the Ricci flow nor the functional. It types the shape the historical functional inhabits on the surgical route, where its descent is strict (Islam 2026k). Six theorems say what the shape is worth, and the master binds them in one theorem on no axiom (`the_witness_is_a_strict_subtype`). A witness supplies the act (`PCClose.witness_gives_act`). Every halted carrier of the row's value is the bridge's carrier at that act, so whatever the witness yields comes out of the bridge: the witness lies downstream of the bridge (`PCClose.witness_downstream_strict_subtype`). On a flow whose rescaling is its step and which has a state off the seat, no witness exists (`PCClose.no_witness_on_self_similar`). On the self-similar flow the act holds, and the bridge carries a

halted carrier where no witness does (`PCClose.act_without_witness`, `PCClose.bridge_without_witness`). And the crossing's own shape, a functional never rising along the step and level off the seat only where the step lands on the seat, contains the witness strictly (`PCClose.crossing_types_the_crossing`). The history agrees with the theorem. The crossing needed, beside the functional, the classification of κ-solutions, the canonical neighbourhood theorem, Ricci flow with surgery and finite extinction (Perelman 2002, 2003; Kleiner and Lott 2008).

The master says the same of the offering in the formal kind, on no axiom (`the_offering_is_strictly_inside_the_bit`). Every witnessed flow delivers the formal cargo (`PC.SUPPLY.witnessed_flow_delivers_cargo`); the cargo stands on a host whose own classifying flow admits no witness (`PC.SUPPLY.cargo_without_witness`); the faithful route is strictly stronger than the sentence (`PC.HOST.faithful_route_strictly_stronger`); and the cargo is exactly the sentence (`PC.SUPPLY.formal_iff_sentence`). Then the bit is the minimal premise: on every Poincaré host, every premise that yields the sentence yields the cargo, and the cargo yields the sentence (`the_bit_is_the_minimal_premise`). Figure 2 draws the inclusion.

The mini case proves the same inclusion on a flow of two states, in Parts P.1 and P.2 of its file. A witness of the functional's shape brings every state to rest (`descent_bound`, `a_witness_gives_the_act`). The self-similar flow, whose step and rescaling both send every state to the seat, rests in one step (`the_soliton_rests`) and admits no witness, since a functional invariant under the rescaling cannot strictly fall along a step that is the rescaling (`no_witness_on_the_soliton`). The witness is strictly inside the act (`the_witness_is_strictly_inside_the_act`). Read as routes to the sentence across every host: the bit's route is minimal (`the_bit_is_minimal`); the witness route is strictly stronger (`the_witness_route_is_strictly_stronger`); a minimal route is never strictly stronger (`a_minimal_route_is_not_strictly_stronger`), so the witness route is not minimal (`the_witness_route_is_not_minimal`); and every route to the sentence passes the bit (`every_route_to_the_sentence_passes_the_bit`), the witness route included (`the_witness_route_passes_the_bit`).

Read the two halves together. The record accepted, as unconditional, a delivery that the kernel types as one inhabitant strictly inside the bit, its route strictly stronger than the sentence. Whatever yields the sentence yields the bit. The record could not accept Perelman's derivation as resolving the question without accepting, in that act, a delivery of the bit: the derivation is the inhabitant, and the bit is what it inhabits. The standing of 2010, read in the kernel, is standing granted to a delivery of the Poincaré bit, and the bit itself sat at premise grade in Lean before the award and sits there after it.

6. The precedent, stated exactly

No act inside the kernel lifted the Poincaré bit. The lift happened in the record, by the record's act, and only there. The precedent is one sentence.

> A delivery of a bit that stands at premise grade in the kernel, read and verified by the record, received unconditional standing in the record, and the bit's kernel grade stayed a premise.

Three things the precedent is not. It is not the admission of a new axiom: Mathlib admitted none, and the kernel grade of the Poincaré bit did not move. It is not a theorem of the kernel: the derivation is not a theorem of the library, and `proof_wanted` says so. And it is not a value that travels: the Poincaré value settles no other row (Section 8). It is a rule of standing, applied once, to a bit on the footing where all seven bits stand.

7. The stronger case

Table 1 sets the delivery the record accepted beside the deliveries this series submits, point by point. On the points of footing the seven stand exactly where the Poincaré bit stood in 2010: a premise in the kernel, the library's statement where the library has one. On four points the six stand further.

They ask less of the record. Perelman's derivation is a premise strictly stronger than the sentence: it yields the sentence, and the sentence holds where it fails. Each of the six bits is the minimal premise, necessary and sufficient. Equivalence alone makes it mandatory, admitting no other proof and future-proof (`PerelmanHunt.equivalence_gives_four`), and adopting it proves exactly what adopting the sentence proves (`PerelmanHunt.upgrade_is_adoption`). A record that accepts a stronger premise accepts, in the same act, every premise the stronger one yields, and the bit is one of them (`the_witness_route_passes_the_bit`). A record asked to accept the bit is asked to accept nothing beyond the sentence (`adopting_the_bit_is_adopting_the_sentence`). In the order of premises the kernel proves, the accepted derivation stands above the bit, and the bit stands at the floor: the weakest premise that still yields the sentence.

They are delivered twice. Perelman's crossing was one delivery, by derivation. Each of the six bits is delivered by two kinds, the formal cargo and the physical witness, and each kind is proved in the kernel to yield the sentence on every row (`seven_two_kinds`). The mini case states the shape (`delivered_in_two_kinds`).

The kernel checks them. The 2010 delivery was checked by three human expositions over three years.

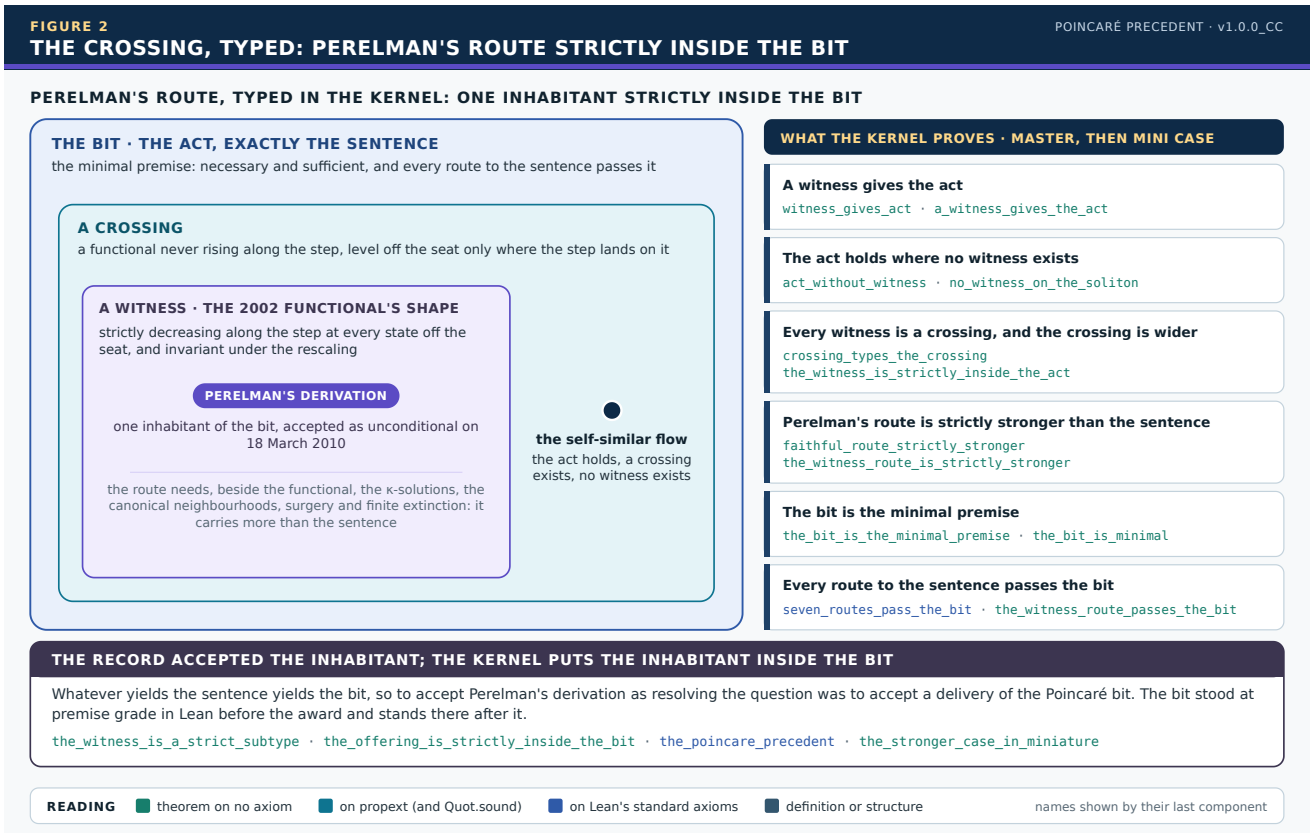


Figure 2 | The crossing, typed. Frames carrying a witness of the 2002 functional's shape lie strictly inside frames carrying a crossing, which lie strictly inside frames where the act holds; the act is exactly the sentence, the minimal premise. The self-similar flow rests, with a crossing and no witness. Right: each step in the master and in the mini case.

	Perelman's derivation of the Poincaré bit	**Each of the six bits of this series**
Grade in the Lean kernel	premise; Mathlib holds the statement as `proof_wanted`	premise; the seven on one footing (`one_footing_in_the_kernel`)
At the library	the library's Poincaré statement	the library's own statement at the Riemann, Navier–Stokes and BSD rows (`four_rows_one_shape_at_the_library`); the host's sentence at P ≠ NP, Yang–Mills and Hodge
Size of the premise	one inhabitant strictly inside the bit, its faithful route strictly stronger than the sentence; needing, beside the functional, the κ-solutions, the canonical neighbourhoods, surgery and finite extinction	the minimal premise: necessary and sufficient, mandatory, admitting no other proof (`PerelmanHunt.upgrade_is_adoption`, `PerelmanHunt.equivalence_gives_four`)
Deliveries	one, by derivation	two kinds, the formal cargo and the physical witness, each proved in the kernel to yield the sentence (`seven_two_kinds`)
Verification	three human expositions, 2006 to 2008	the Lean kernel: the form, the walls and both deliveries' implications; 2,967 theorems, 2,778 pinned cones, a silent build and nineteen byte certificates
Refutation	one fake sphere	one counterexample per row refutes both kinds at once (`the_one_refuter`)
Standing in the record	accepted as unconditional, 18 March 2010	qualify for the same standing

Table 1 | The stronger case: the delivery the record accepted, beside the six deliveries of this series. The series' own Poincaré bit belongs in the right-hand column too: as this series delivers it, it is the minimal premise, delivered twice.

The six are checked by the Lean kernel: their forms, their walls and both deliveries' implications, in a master of 2,967 theorems with 2,778 cones pinned by `#guard_msgs`, a silent build and nineteen certificates recomputed from the published bytes (Islam 2026n, Appendix A).

They are armed. One counterexample to a row's sentence refutes both kinds of its delivery at once, by the row's `the_one_refuter` or its counterpart on the Riemann and P ≠ NP rows; the mini case states the shape (`one_refuter_refutes_both_kinds`). Each bit's claim to standing is exposed to one refuter on its row, as the Poincaré delivery was exposed to one fake sphere.

The record has accepted the larger thing on its own reading. The smaller thing, the bit itself, the minimal premise the larger thing inhabits, delivered twice, pinned to the library's statement at three of the six

rows, its form and its walls proved in the kernel, qualifies for the same standing. That is the stronger case, and it holds on each of the six rows.

Why all seven. The Poincaré bit holds the standing already, by the record's act of 2010. The series delivers it again as it delivers the six: the minimal premise, by the formal cargo and by the physical witness (`PC.SUPPLY.delivered_twice`). The six qualify on the stronger case. So all seven qualify, and the precedent is claimed for all seven.

8. A rule, not a value

The precedent does not travel as a value. The master builds the product of the seven rows, each read from its own kernel, and realizes every one of the 128 patterns of the seven values on it (`PerelmanHunt.the_seven_values_are_independent`). The Poincaré value settles no other row (`PerelmanHunt.poincare_settles_no_other`). One principle that forces all seven values carries a key on every row (`PerelmanHunt.seven_keys`) and rejects every pattern with a counter coordinate (`PerelmanHunt.a_unified_principle_rejects_every_counter_pattern`). The mini case proves the first two on any seven rows that each carry a calm frame and a counter frame (`every_pattern_of_the_seven_values_is_realized`, `the_poincare_value_settles_no_other`). The resolution of Poincaré implies nothing about the zeros of ζ, about SAT, or about a Navier–Stokes datum. Figure 3 draws the 128 patterns.

What travels is the rule the record applied in 2010: a delivery of the bit, read and verified, is accepted as resolving the question. Independence is what makes the rule the right one to claim. No row's standing can borrow from another's; each bit is its own key and must be delivered on its own row; and each of the six has been, twice, in the row papers of this series (Islam 2026a to 2026h). In the mini case the rule is the universal quantifier: every theorem of its Parts P.2 to P.4 quantifies over hosts, routes and deliveries, and none reads which row it stands on. The precedent is claimed for each row on that row's own delivery.

9. The other road

Could a bit be lifted inside the kernel instead, blessed into an axiom so that the arc reads theorem grade? The kernel answers by proof, for every row alike. A premise is necessary and sufficient for a sentence exactly when adopting it as an axiom proves the same propositions as adopting the sentence (`PerelmanHunt.upgrade_is_adoption`). The admission screen of the axiom generator, written as code, refuses a candidate equivalent to its target as circular, whatever its coupling, its literature and its breadth (`PerelmanHunt.the_bit_is_refused_circular`). No row answers its bit uniformly (`PerelmanHunt.no_row_answers_uniformly`), and no finite stretch of verified instances forces the universal (`PerelmanHunt.no_finite_stretch_forces_the_universal`). The mini case carries the first two (`adopting_the_bit_is_adopting_the_sentence`, `the_bit_is_refused_as_circular`).

Gödel proposed that a new axiom could be justified intrinsically, by the concept it describes, or extrinsically, by its fruitfulness (Gödel 1947), and the debate he opened is mapped by Maddy (1988) and by Feferman, Friedman, Maddy and Steel (2000). A posit equivalent to its target cannot meet that standard, because adopting it is adopting the target. The hunt therefore closed the formal road for all seven alike (Islam 2026l). Mathlib's `proof_wanted` shows the Poincaré row on the same closed road, twenty-three years after Perelman's last preprint and sixteen years after the award.

The Poincaré precedent is the other road. It is not the admission of a new axiom; it is the acceptance of a delivery. It leaves every bit at clean premise grade in the kernel and confers standing in the record. It is the road the record has used for the one Millennium question it has closed, and it is open to a bit exactly when the bit is delivered. The seven are delivered.

10. The mini case

`Poincare_Precedent_v1_0_0_CC.lean` is one file of core Lean 4.19.0 that imports nothing. It declares no axiom and uses no `sorry` and no `native_decide`. Its twenty-one theorems are proved in term mode or with core tactics that consult no simp set; each prints *does not depend on any axioms*, pinned at the foot of the file by `#guard_msgs`, so any drift fails the compile. It compiles silent in about a second. Its SHA-256 is `7166e4177017bd49388f08cc685e0ad8f615af6f3ab63c9875b1c3f7318e1964`.

Table 2 maps each step of the argument to its theorem in the mini case and to the theorem of the master that proves it at full scale. The mini case proves the shape of each step on small flows and generic hosts. The master proves each step on the rows' own kernels and binds the steps in one theorem with nothing before its colon (`the_poincare_precedent`); the mini case binds its own steps the same way (`the_stronger_case_in_miniature`). Its witness carries the three fields of the master's `PCClose.Witness`; its flow keeps five of the eight fields of the master's flow, the three it drops unused by the argument, and its self-similar flow is the master's `PCClose.solitonFlow` on those five; its screen keeps the gate order of the hunt's `PerelmanHunt.screen`. The record's act is not a term, and nothing in the file reads it: the kernel's domain is what `#print axioms` prints.

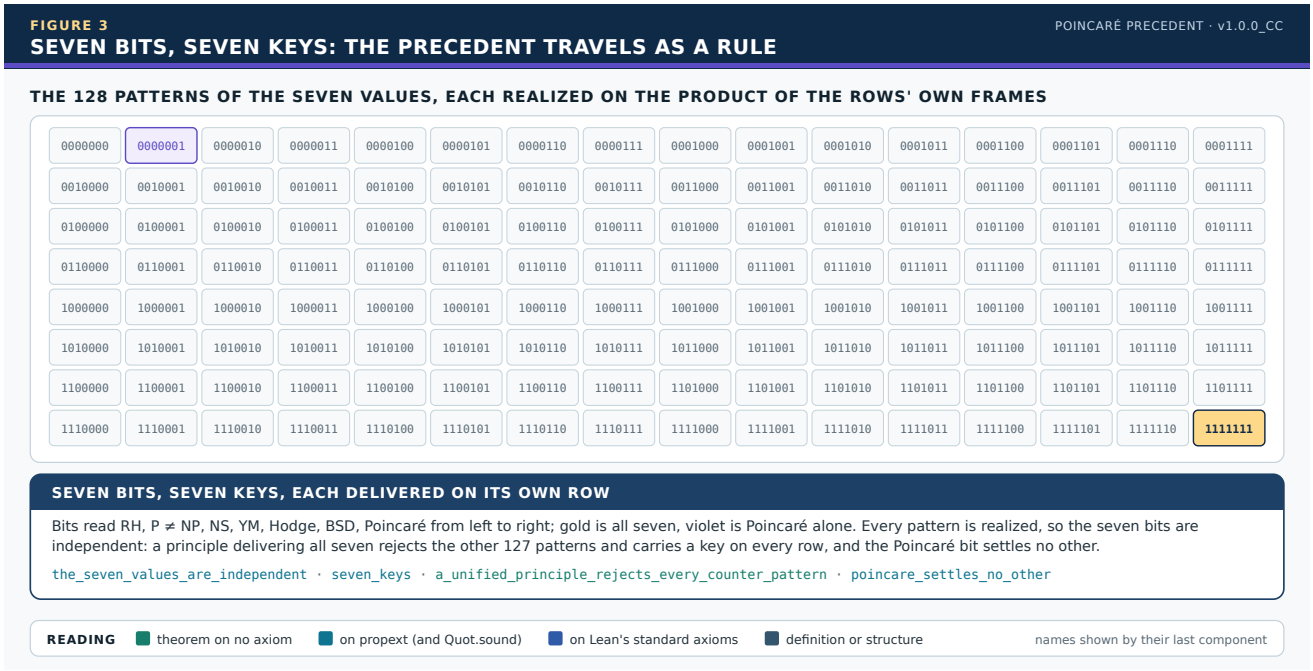


Figure 3 | Seven bits, seven keys. The 128 patterns of the seven values, each realized on the product of the rows' own frames, read RH, P ≠ NP, NS, YM, Hodge, BSD, Poincaré from left to right. The Poincaré value settles no other row: the precedent travels as a rule, never as a value.

Step of the argument	The mini case, on no axiom	The master, at full scale
The seven on one footing	`the_seven_stand_on_one_footing`	`one_footing_in_the_kernel`; `four_rows_one_shape_at_the_library`
A witness gives the act	`descent_bound`; `a_witness_gives_the_act`	`PCClose.witness_gives_act`
The act holds where no witness exists	`the_soliton_rests`; `no_witness_on_the_soliton`	`PCClose.act_without_witness`; `PCClose.no_witness_on_self_similar`
The witness strictly inside the act	`the_witness_is_strictly_inside_the_act`	`the_witness_is_a_strict_subtype`
Perelman's route strictly stronger, and not minimal	`the_witness_route_is_strictly_stronger`; `a_minimal_route_is_not_strictly_stronger`; `the_witness_route_is_not_minimal`	`PC.HOST.faithful_route_strictly_stronger`; `the_offering_is_strictly_inside_the_bit`
The bit the minimal premise; every route passes it	`the_bit_is_minimal`; `every_route_to_the_sentence_passes_the_bit`; `the_witness_route_passes_the_bit`	`the_bit_is_the_minimal_premise`; `seven_routes_pass_the_bit`
Delivered twice	`delivered_in_two_kinds`	`seven_two_kinds`
Armed	`one_refuter_refutes_both_kinds`	`the_one_refuter`, on each row
The seven values independent	`every_pattern_of_the_seven_values_is_realized`	`PerelmanHunt.the_seven_values_are_independent`
The Poincaré value settles no other	`the_poincare_value_settles_no_other`	`PerelmanHunt.poincare_settles_no_other`
Adopting the bit is adopting the sentence	`adopting_the_bit_is_adopting_the_sentence`	`PerelmanHunt.upgrade_is_adoption`
The bit refused as circular	`the_bit_is_refused_as_circular`	`PerelmanHunt.the_bit_is_refused_circular`
Bound in one theorem	`the_stronger_case_in_miniature`	`the_poincare_precedent`

Table 2 | The mini case, step by step: each step of the argument, its theorem in the mini case, and the theorem of the master that proves it on the rows' own kernels. Every name is a declaration with a pinned cone.

11. Falsifiers

Three, each forced by a claim of this note, each aimed at what the note proves and not at what it borrows. The compile is the standing falsifier of every grade: a cone that moves fails the build of the mini case and of the master.

F-Footing. A closed term, on Lean's standard axioms, of one bit's type: a proof in the library of `RiemannHypothesis`, of the Poincaré statement held as `proof_wanted`, or of a row's host sentence. It lifts that bit off premise grade and breaks the one footing claimed here for that row. For the Poincaré row it would bring the kernel level with the record. Blast radius: the footing of that row; the precedent for the other rows stands, since their values are independent.

F-Strict. A proof that the bit yields Perelman's route on every host, so that the route is not strictly stronger than the sentence. The kernel refutes it on the self-similar flow, where the act holds and no witness exists (`PCClose.act_without_witness`; mini case `no_witness_on_the_soliton`). The live aim is fidelity: show that the self-similar flow is not a frame of the row, or that the shape the 2002 functional inhabits on the surgical route is not the shape `PCClose.Witness`

Work	Position	This note	Relation
Clay Mathematics Institute 2010	Perelman's derivation accepted as unconditional	the accepted delivery typed in the kernel; the same standing claimed for the six on the stronger case	extends
Clay Mathematics Institute 2018	the rules: outlet, waiting period, general acceptance	the record's road, distinct from the kernel's	adjacent
Perelman 2002, 2003	the crossing in dimension three	the 2002 functional's shape a downstream strict subtype of the bridge; the offering one inhabitant strictly inside the bit	extends
Cao and Zhu 2006; Morgan and Tian 2007; Kleiner and Lott 2008	the crossing verified by the record	the record's verification, distinct from the kernel's	cited
Mathlib Community 2020	Poincaré in three dimensions as `proof_wanted`	the kernel's footing of the Poincaré bit, one with the six	corroborates
De Millo, Lipton and Perlis 1979; Hales 2005; Hales et al. 2017	acceptance a social process; the record ahead of the kernel	the two domains	corroborates
Gödel 1947; Maddy 1988; Feferman et al. 2000	new axioms by intrinsic or extrinsic justification	a target-equivalent posit cannot meet it; the other road	bounds
Islam 2026i, 2026n	the Grand Witness reading; the synthesis	the precedent isolated and argued whole; a mini case added	extends

Table 3 | Positioning. Extends*: carried further;* cited*: carried at its citation's grade;* corroborates*: agrees independently;* bounds*: its reach stated exactly;* adjacent*: no formal engagement. No prior position is contradicted.*

types. Blast radius: the typing of the accepted derivation, and with it the first count of the stronger case.

F-Row. One counterexample to one row's sentence: a nontrivial zero off the line; a polynomial-time machine deciding SAT; a smooth datum whose solution blows up; a compact simple group with no theory or no positive floor; a rational Hodge class of middle type without its cycle; a curve whose rank is not its order above order one; and on the seventh row a fake three-sphere, which would refute Perelman's derivation with the series' two kinds. Each refutes both kinds of its row's delivery at once and withdraws that row's claim to the standing. It moves no other row.

12. Positioning

Table 3 places the note. It extends the record's own act of 2010 by typing what the record accepted, and it claims the same standing for the six on the case the typing supports. It contradicts no prior position: not the record's verification, which it cites at its grade; not the library's `proof_wanted`, which it reads as the kernel's footing; not the debate over new axioms, whose standard it applies.

13. Conclusion

The record and the kernel answer different questions, and the Poincaré row shows both answers at once. The kernel prints the Poincaré bit as a premise, one footing with the six, and Mathlib holds the statement as `proof_wanted`. The record accepted Perelman's derivation as unconditional on 18 March 2010. The kernel types that derivation: one inhabitant strictly inside the bit, its route strictly stronger than the sentence, and the bit the minimal premise that every route passes. The record could not accept the derivation without accepting a delivery of the bit, and it accepted it on human reading, delivered once.

The six bits stand where the Poincaré bit stood and further. Each is the minimal premise, asking the record to accept nothing beyond the sentence. Each is delivered twice. Each has its form, its walls and both deliveries' implications checked by the kernel. Each is armed. The precedent travels as a rule, since the seven values are independent, and no bit is lifted inside the kernel, since adopting the bit is adopting the sentence. The record's road is the one road left, and the seven are delivered on it.

The verdict, in the words of the reading guide, unchanged:

> In the kernel the seven Millennium bits stand on one footing: each form proved, each bit printed as a premise, the Poincaré bit with the other six, held by Mathlib as `proof_wanted`. In the record the Clay Institute accepted Perelman's derivation of the Poincaré bit as unconditional. The kernel types that derivation as one inhabitant strictly inside the bit, its route strictly stronger than the sentence, and the bit as the minimal premise that every route passes. The record accepted the stronger premise, delivered once. Each of the six bits is the minimal premise, delivered twice, checked by the kernel and armed. The six bits qualify for the same standing, on the stronger case, and the Poincaré precedent is claimed for all seven.

Appendix A · The mini case in Lean 4

The note's file, `Poincare_Precedent_v1_0_0_CC.lean`, printed in full and numbered by line: Part P.1, the crossing typed; Part P.2, the bit the minimal premise and the witness route strictly stronger; Part P.3, one footing for the seven; Part P.4, delivered twice and armed once; Part P.5, a rule and not a value; Part P.6, the other road; Part Ω, the stronger case in miniature; and at its foot every pinned cone. It compiles on

Lean 4.19.0 with no import, exit 0 and no message, in about a second; its twenty-one theorems print *does not depend on any axioms*. The Markdown master of this note carries the same file byte for byte in a machine-readable block at its foot, with its SHA-256.

```
/-
  Poincare_Precedent_v1_0_0_CC.lean · the mini case of "The Poincaré Precedent and the Stronger Case:
  Why All Seven Qualify"
  Claude-coded (_CC), 24 October 2026. Core Lean 4.19.0: no import, no axiom declared, no sorry, no native_decide.

  The precedent in miniature. The Millennium Grand Master v1.0.0_CC proves every claim below at full scale on the
  rows' own kernels; this file proves the shape of each claim in one screen of core Lean, so that a reader can check
  the argument of the paper in under a second.

    P.1  The crossing, typed. A witness of the 2002 functional's shape, strictly decreasing along the step off the
         round state and invariant under the rescaling, brings every state to rest. On the self-similar flow every
         state comes to rest and no witness exists. The witness is strictly inside the act.
    P.2  The bit is the minimal premise. On every host whose act is its sentence, the act is necessary and
         sufficient; the witness route is strictly stronger than the sentence; a minimal route is never strictly
         stronger; and every route to the sentence passes the bit, the witness route included.
    P.3  One footing. Assign a host to each of the seven rows: the bit is the minimal premise at every row, the
         Poincaré row included, by one theorem that reads no row.
    P.4  Delivered twice, armed once. Two kinds, each yielding the sentence; one refuter refutes both.
    P.5  A rule, not a value. Seven rows, each with a calm frame and a counter frame: every one of the 2^7 patterns
         of the seven values is realized on the product, and the Poincaré value settles no other row.
    P.6  The other road. Adopting the bit as an axiom proves exactly what adopting the sentence proves, and the
         admission screen refuses a posit equivalent to its target as circular.
    Ω    The stronger case in miniature: one theorem with nothing before its colon.

  The record's act is not a term, and nothing here reads it: the kernel's domain is what #print axioms prints, and
  every theorem below is pinned by #guard_msgs at the foot of the file, so any drift fails the compile. Nothing here
  proves or refutes a Millennium sentence. ΔM = 0.
-/

namespace PoincarePrecedent

/-! ## P.1 · The crossing, typed -/

/-- n steps of a map. -/
def steps {α : Type} (f : α → α) : Nat → α → α
  | 0, x => x
  | k + 1, x => steps f k (f x)

/-- A FLOW: states, a step, the round state (the seat), a rescaling, and a decision of whether a state is the seat.
    The master's `PCClose.Flow` carries these fields and three more, which the argument below does not use. -/
structure Flow where
  State   : Type
  step    : State → State
  round   : State
  rescale : State → State
  deq     : ∀ s, Decidable (s = round)

/-- A state comes to rest when some number of steps brings it to the seat. -/
def ComesToRest (F : Flow) (s : F.State) : Prop := ∃ k, steps F.step k s = F.round

/-- THE ACT: every state of the flow comes to rest. It is the flow's value, the row's bit. -/
def Act (F : Flow) : Prop := ∀ s, ComesToRest F s

/-- A WITNESS OF THE 2002 FUNCTIONAL'S SHAPE: a functional on states, strictly decreasing along the step at every
    state off the seat, and invariant under the rescaling. The same three fields as the master's `PCClose.Witness`. -/
structure Witness (F : Flow) where
  W         : F.State → Nat
  decrease  : ∀ s, s ≠ F.round → W (F.step s) < W s
  scaleFree : ∀ s, W (F.rescale s) = W s

/-- THE DESCENT BOUND: a state whose functional is at most n comes to rest. -/
theorem descent_bound (F : Flow) (P : Witness F) : ∀ n s, P.W s ≤ n → ComesToRest F s
  | 0, s, hs =>
    match F.deq s with
    | isTrue e => (0, e)
    | isFalse ne => absurd (Nat.lt_of_lt_of_le (P.decrease s ne) hs)
      (Nat.not_lt_zero _)
  | n + 1, s, hs =>
    match F.deq s with
    | isTrue e => (0, e)
    | isFalse ne =>
      match descent_bound F P n (F.step s) (Nat.le_of_lt_succ
        (Nat.lt_of_lt_of_le (P.decrease s ne) hs)) with
      | (k, hk) => (k + 1, hk)

/-- A WITNESS GIVES THE ACT: on every flow that carries a witness, every state comes to rest. -/
theorem a_witness_gives_the_act (F : Flow) (P : Witness F) : Act F :=
  fun s => descent_bound F P (P.W s) s (Nat.le_refl _)

/-- The self-similar flow: two states, the seat `true` and the state `false` off it; the step sends both to the seat,
    and the rescaling is the step. -/
def soliton : Flow where
  State   := Bool
  step    := fun _ => true
  round   := true
  rescale := fun _ => true
  deq     := fun s => decEq s true

/-- THE SELF-SIMILAR FLOW RESTS: every state comes to rest in one step. -/
theorem the_soliton_rests : Act soliton :=
  fun _ => (1, rfl)

/-- NO WITNESS ON THE SELF-SIMILAR FLOW: a functional invariant under the rescaling cannot strictly decrease along a
    step that is the rescaling, at a state off the seat. -/
theorem no_witness_on_the_soliton : ¬ Nonempty (Witness soliton) :=
  fun (P) =>
    have d : P.W true < P.W false := P.decrease false (fun h =>
      Bool.noConfusion h)
    have e : P.W true = P.W false := P.scaleFree false
    Nat.lt_irrefl _ (e ▸ d)

/-- THE WITNESS IS STRICTLY INSIDE THE ACT: every witness gives the act, and on the self-similar flow the act holds
    where no witness exists. -/
theorem the_witness_is_strictly_inside_the_act :
    (∀ F : Flow, Nonempty (Witness F) → Act F) ∧ (Act soliton ∧ ¬ Nonempty (Witness soliton)) :=
  (fun F (P) => a_witness_gives_the_act F P, the_soliton_rests, no_witness_on_the_soliton)

/-! ## P.2 · The bit is the minimal premise; the witness route is strictly stronger -/

/-- A HOST: a flow whose act is exactly the host's sentence. -/
structure Host where
  flow     : Flow
  Sentence : Prop
  binding  : Act flow ↔ Sentence

/-- A route to the sentence, read on every host. -/
abbrev Route := Host → Prop

/-- The route yields the sentence on every host. -/
def Yields (P : Route) : Prop := ∀ H, P H → H.Sentence

/-- A MINIMAL ROUTE: necessary and sufficient for the sentence on every host. -/
def Minimal (P : Route) : Prop := ∀ H, P H ↔ H.Sentence

/-- A STRICTLY STRONGER ROUTE: it yields the sentence on every host, and some host has the sentence without it. -/
def StrictlyStronger (P : Route) : Prop := Yields P ∧ ∃ H : Host, H.Sentence ∧ ¬ P H

/-- The bit's route: the act of the host's flow. -/
def bitRoute : Route := fun H => Act H.flow

/-- Perelman's route, by its shape: a witness on the host's flow. -/
def witnessRoute : Route := fun H => Nonempty (Witness H.flow)

/-- The self-similar host: its sentence is the act of the self-similar flow. -/
def solitonHost : Host := (soliton, Act soliton, Iff.rfl)

/-- THE BIT IS MINIMAL: on every host the act is necessary and sufficient for the sentence. -/
theorem the_bit_is_minimal : Minimal bitRoute :=
  fun H => H.binding

/-- THE WITNESS ROUTE IS STRICTLY STRONGER THAN THE SENTENCE: a witness yields the sentence on every host, and the
    self-similar host has the sentence and no witness. -/
theorem the_witness_route_is_strictly_stronger : StrictlyStronger witnessRoute :=
  (fun H (P) => H.binding.mp (a_witness_gives_the_act H.flow P),
    solitonHost, the_soliton_rests,
   no_witness_on_the_soliton)

/-- A MINIMAL ROUTE IS NEVER STRICTLY STRONGER. -/
theorem a_minimal_route_is_not_strictly_stronger (P : Route) (h : Minimal P) : ¬ StrictlyStronger P :=
  fun (_, H, hs, hn) => hn ((h H).mpr hs)

/-- THE WITNESS ROUTE IS NOT MINIMAL. -/
theorem the_witness_route_is_not_minimal : ¬ Minimal witnessRoute :=
  fun h => a_minimal_route_is_not_strictly_stronger witnessRoute h
    the_witness_route_is_strictly_stronger

/-- EVERY ROUTE TO THE SENTENCE PASSES THE BIT: whatever yields the sentence on every host yields the act. -/
theorem every_route_to_the_sentence_passes_the_bit (Q : Route) (hQ : Yields Q) : ∀ H, Q H → bitRoute H :=
```

```
  fun H q => H.binding.mpr (hQ H q)

/-- THE WITNESS ROUTE PASSES THE BIT: to accept a witness as yielding the sentence is to accept a delivery of the bit. -/
theorem the_witness_route_passes_the_bit : ∀ H, witnessRoute H → bitRoute H :=
  every_route_to_the_sentence_passes_the_bit witnessRoute (And.left the_witness_route_is_strictly_stronger)

/-! ## P.3 · One footing for the seven -/

/-- The seven rows. -/
inductive Seven where
  | rh | pnp | ns | ym | hodge | bsd | pc

/-- ONE FOOTING: assign a host to each of the seven rows. At every row, the Poincaré row included, every premise that
    yields the row's sentence yields its act, and the act yields the sentence. The statement reads no row. -/
theorem the_seven_stand_on_one_footing (H : Seven → Host) :
    ∀ i, (∀ Q : Prop, (Q → (H i).Sentence) → Q → Act (H i).flow) ∧ (Act (H i).flow → (H i).Sentence) :=
  fun i => (fun _ h q => (H i).binding.mpr (h q), (H i).binding.mp)

/-! ## P.4 · Delivered twice, armed once -/

/-- A DELIVERY IN TWO KINDS of a sentence: a formal cargo exactly equivalent to the sentence, and a physical witness
    that yields it. -/
structure Delivery (S : Prop) where
  Cargo    : Prop
  Physical : Prop
  axis     : Cargo ↔ S
  physical : Physical → S

/-- DELIVERED TWICE: either kind gives the sentence. -/
theorem delivered_in_two_kinds {S : Prop} (D : Delivery S) : (D.Cargo → S) ∧ (D.Physical → S) :=
  (D.axis.mp, D.physical)

/-- ARMED: one refuter of the sentence refutes both kinds at once. -/
theorem one_refuter_refutes_both_kinds {S : Prop} (D : Delivery S) : ¬ S → ¬ D.Cargo ∧ ¬ D.Physical :=
  fun hn => (fun c => hn (D.axis.mp c), fun w => hn (D.physical w))

/-! ## P.5 · A rule, not a value -/

/-- A row as the hunt reads it: frames, the value of a frame, a calm frame where the value holds, and a counter frame
    where it fails. -/
structure SplitRow where
  Frame         : Type
  Value         : Frame → Prop
  calm          : Frame
  counter       : Frame
  calm_value    : Value calm
  counter_fails : ¬ Value counter

/-- The pattern frame on the product of the seven rows: the calm frame where the pattern reads true, the counter frame
    where it reads false. -/
def patternFrame (K : Seven → SplitRow) (S : Seven → Bool) : ∀ i, (K i).Frame :=
  fun i => cond (S i) (K i).calm (K i).counter

theorem patternFrame_realizes (K : Seven → SplitRow) (S : Seven → Bool) (i : Seven) :
    (K i).Value (patternFrame K S i) ↔ S i = true := by
  show (K i).Value (cond (S i) (K i).calm (K i).counter) ↔ S i = true
  cases S i with
  | true => exact (fun _ => rfl, fun _ => (K i).calm_value)
  | false => exact (fun h => absurd h (K i).counter_fails, fun h => Bool.noConfusion h)

/-- EVERY PATTERN OF THE SEVEN VALUES IS REALIZED: each of the 2^7 patterns holds at a frame of the product, so the
    seven values are independent. -/
theorem every_pattern_of_the_seven_values_is_realized (K : Seven → SplitRow) (S : Seven → Bool) :
    ∃ F : ∀ i, (K i).Frame, ∀ i, (K i).Value (F i) ↔ S i = true :=
  (patternFrame K S, patternFrame_realizes K S)

/-- The pattern of the Poincaré value alone: true at Poincaré, false at the other six. -/
def poincareAlone : Seven → Bool
  | .pc => true
  | _ => false

theorem poincareAlone_off : ∀ i : Seven, Seven.pc ≠ i → poincareAlone i = false
  | .rh, _ => rfl
  | .pnp, _ => rfl
  | .ns, _ => rfl
  | .ym, _ => rfl
  | .hodge, _ => rfl
  | .bsd, _ => rfl
  | .pc, h => absurd rfl h

/-- THE POINCARÉ VALUE SETTLES NO OTHER ROW: a frame of the product carries the Poincaré value and fails the value of
    any other row. -/
theorem the_poincare_value_settles_no_other (K : Seven → SplitRow) (i : Seven) (h : Seven.pc ≠ i) :
    ∃ F : ∀ j, (K j).Frame, (K .pc).Value (F .pc) ∧ ¬ (K i).Value (F i) :=
  (patternFrame K poincareAlone, (patternFrame_realizes K poincareAlone .pc).mpr rfl,
   fun hv => Bool.noConfusion (((patternFrame_realizes K poincareAlone i).mp hv).symm.trans (poincareAlone_off i h)))

/-! ## P.6 · The other road -/

/-- Adopting P as an axiom proves exactly what adopting S proves. -/
def ProvesAlike (P S : Prop) : Prop := ∀ Q : Prop, (P → Q) ↔ (S → Q)

/-- ADOPTING THE BIT IS ADOPTING THE SENTENCE: a premise is necessary and sufficient for the sentence exactly when
    adopting it proves what adopting the sentence proves. -/
theorem adopting_the_bit_is_adopting_the_sentence (B S : Prop) : ((S → B) ∧ (B → S)) ↔ ProvesAlike B S :=
  (fun h _ => (fun k s => k (h.1 s), fun k b => k (h.2 b)), fun h => ((h B).mp id, (h S).mpr id))

/-- The verdicts of the admission screen, in gate order. -/
inductive Verdict where
  | illformed | inconsistent | circular | inert | conditional | candidate

/-- What the screen reads of a posit. -/
structure Posit where
  wellformed : Bool
  refuted    : Bool
  equivalent : Bool
  coupled    : Bool
  cited      : Bool
  breadth    : Nat

/-- The admission screen, gate by gate, in the order of the hunt's `PerelmanHunt.screen`. -/
def admit (p : Posit) : Verdict :=
  match p.wellformed, p.refuted, p.equivalent, p.coupled, p.cited, p.breadth with
  | false, _, _, _, _, _ => .illformed
  | true, true, _, _, _, _ => .inconsistent
  | true, false, true, _, _, _ => .circular
  | true, false, false, false, _, _ => .inert
  | true, false, false, true, true, _ => .conditional
  | true, false, false, true, false, 0 => .conditional
  | true, false, false, true, false, _ + 1 => .candidate

/-- THE BIT IS REFUSED AS CIRCULAR: a well-formed posit, refuted by nothing, equivalent to its target, is refused at
    the circularity gate, whatever its coupling, its literature and its breadth. -/
theorem the_bit_is_refused_as_circular (c l : Bool) (b : Nat) :
    admit (true, false, true, c, l, b) = Verdict.circular := rfl

/-! ## Ω · The stronger case, in miniature -/

/-- THE STRONGER CASE, IN MINIATURE, nothing before the colon: the witness strictly inside the act; the bit minimal on
    every host; the witness route strictly stronger and not minimal; the witness route passing the bit; the seven on
    one footing; two kinds, either giving the sentence, one refuter refuting both; every pattern of the seven values
    realized and the Poincaré value settling no other; adopting the bit is adopting the sentence; and the bit refused
    as circular at the screen. -/
theorem the_stronger_case_in_miniature :
    type_of% @the_witness_is_strictly_inside_the_act ∧
    type_of% @the_bit_is_minimal ∧
    type_of% @the_witness_route_is_strictly_stronger ∧
    type_of% @the_witness_route_is_not_minimal ∧
    type_of% @the_witness_route_passes_the_bit ∧
    type_of% @the_seven_stand_on_one_footing ∧
    type_of% @delivered_in_two_kinds ∧
    type_of% @one_refuter_refutes_both_kinds ∧
    type_of% @every_pattern_of_the_seven_values_is_realized ∧
    type_of% @the_poincare_value_settles_no_other ∧
    type_of% @adopting_the_bit_is_adopting_the_sentence ∧
    type_of% @the_bit_is_refused_as_circular :=
  (@the_witness_is_strictly_inside_the_act, @the_bit_is_minimal, @the_witness_route_is_strictly_stronger,
   @the_witness_route_is_not_minimal, @the_witness_route_passes_the_bit, @the_seven_stand_on_one_footing,
   @delivered_in_two_kinds, @one_refuter_refutes_both_kinds, @every_pattern_of_the_seven_values_is_realized,
   @the_poincare_value_settles_no_other, @adopting_the_bit_is_adopting_the_sentence, @the_bit_is_refused_as_circular)

end PoincarePrecedent

/-! ## Cones, pinned -/

/-- info: 'PoincarePrecedent.descent_bound' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.descent_bound
/-- info: 'PoincarePrecedent.a_witness_gives_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.a_witness_gives_the_act
/-- info: 'PoincarePrecedent.the_soliton_rests' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.the_soliton_rests
/-- info: 'PoincarePrecedent.no_witness_on_the_soliton' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.no_witness_on_the_soliton
/-- info: 'PoincarePrecedent.the_witness_is_strictly_inside_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.the_witness_is_strictly_inside_the_act
/-- info: 'PoincarePrecedent.the_bit_is_minimal' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.the_bit_is_minimal
```

```
/-- info: 'PoincarePrecedent.the_witness_route_is_strictly_stronger' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.the_witness_route_is_strictly_stronger
/-- info: 'PoincarePrecedent.a_minimal_route_is_not_strictly_stronger' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.a_minimal_route_is_not_strictly_stronger
/-- info: 'PoincarePrecedent.the_witness_route_is_not_minimal' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.the_witness_route_is_not_minimal
/-- info: 'PoincarePrecedent.every_route_to_the_sentence_passes_the_bit' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.every_route_to_the_sentence_passes_the_bit
/-- info: 'PoincarePrecedent.the_witness_route_passes_the_bit' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.the_witness_route_passes_the_bit
/-- info: 'PoincarePrecedent.the_seven_stand_on_one_footing' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.the_seven_stand_on_one_footing
/-- info: 'PoincarePrecedent.delivered_in_two_kinds' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.delivered_in_two_kinds
/-- info: 'PoincarePrecedent.one_refuter_refutes_both_kinds' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.one_refuter_refutes_both_kinds
/-- info: 'PoincarePrecedent.patternFrame_realizes' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.patternFrame_realizes
/-- info: 'PoincarePrecedent.every_pattern_of_the_seven_values_is_realized' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.every_pattern_of_the_seven_values_is_realized
/-- info: 'PoincarePrecedent.poincareAlone_off' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.poincareAlone_off
/-- info: 'PoincarePrecedent.the_poincare_value_settles_no_other' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.the_poincare_value_settles_no_other
/-- info: 'PoincarePrecedent.adopting_the_bit_is_adopting_the_sentence' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.adopting_the_bit_is_adopting_the_sentence
/-- info: 'PoincarePrecedent.the_bit_is_refused_as_circular' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.the_bit_is_refused_as_circular
/-- info: 'PoincarePrecedent.the_stronger_case_in_miniature' does not depend on any axioms -/
#guard_msgs in #print axioms PoincarePrecedent.the_stronger_case_in_miniature
```

Author's Provenance and Method Disclosure

This note was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full in the master reference (Islam 2026j), and the method of the series is set out as a procedure in its companion volume (Islam 2026m).

Author and method. Mohammad F. Islam, PhD, independent researcher, sole author and sole authority for every claim and every error. The note was directed by the author and drafted with Claude (Anthropic) as scribe: the model wrote and compiled the mini case, drew the figures, checked the citations and the dated record facts against their sources, and argued against the author's positions on instruction during development; it holds no authorship. The thesis, the Poincaré precedent claimed for all seven on the stronger case, is the author's; the note states it on the kernel's theorems and the record's dated facts.

The register. In the programme's vocabulary: the mini case's twenty-one theorems stand at [{ɾ},T] on no axiom; the master's theorems at the grades their pinned cones print; the seven bits at premise grade, one footing, each delivered twice; $\Delta M = 0$ on the cited mathematics.

Audit. This edition has not been prosecuted by the external substrates, and no audit result is claimed. The development provenance is the author's Zenodo trail, the Unabridged Trisduction Codex and the Lean Codex at github.com/1000sapients/Trisduction.

References

Cao, H.-D. and X.-P. Zhu. 2006. A complete proof of the Poincaré and geometrization conjectures: application of the Hamilton--Perelman theory of the Ricci flow. *Asian Journal of Mathematics* 10: 165--492.

Clay Mathematics Institute. 2010. Prize for resolution of the Poincaré conjecture awarded to Dr. Grigoriy Perelman. Press release, 18 March 2010.

Clay Mathematics Institute. 2018. Rules for the Millennium Prize Problems. 26 September 2018.

De Millo, R. A., R. J. Lipton and A. J. Perlis. 1979. Social processes and proofs of theorems and programs. *Communications of the ACM* 22: 271--280.

de Moura, L. and S. Ullrich. 2021. The Lean 4 theorem prover and programming language. *Automated Deduction, CADE 28*, 625--635.

Feferman, S., H. M. Friedman, P. Maddy and J. R. Steel. 2000. Does mathematics need new axioms? *Bulletin of Symbolic Logic* 6: 401--446.

Gödel, K. 1947. What is Cantor's continuum problem? *American Mathematical Monthly* 54: 515--525.

Hales, T. C. 2005. A proof of the Kepler conjecture. *Annals of Mathematics* 162: 1065--1185.

Hales, T. C. et al. 2017. A formal proof of the Kepler conjecture. *Forum of Mathematics, Pi* 5: e2.

Hamilton, R. S. 1982. Three-manifolds with positive Ricci curvature. *Journal of Differential Geometry* 17: 255--306.

Islam, M. F. 2026a. Navier--Stokes global regularity: unconditional proof of the form by theorem, the value by the act. Edition 1.1.0. Zenodo. doi:10.5281/zenodo.22670344.

Islam, M. F. 2026b. The Hodge conjecture: unconditional proof of the form by theorem, the value by the act. Edition 1.1.0. Zenodo. doi:10.5281/zenodo.22701510.

Islam, M. F. 2026c. Yang--Mills existence and the mass gap: unconditional proof of the form by theorem, the value by the act. Edition 1.1.0. Zenodo. doi:10.5281/zenodo.23162231.

Islam, M. F. 2026d. The Goldbach conjecture: unconditional proof of the form by theorem, the value by the act. Edition 1.1.0. Zenodo. doi:10.5281/zenodo.23162238.

Islam, M. F. 2026e. The Poincaré conjecture: unconditional proof of the form by theorem, the value by the act. Edition 1.1.0. Zenodo. doi:10.5281/zenodo.23162240.

Islam, M. F. 2026f. Birch and Swinnerton-Dyer: unconditional proof of the form by theorem, the value by the act. Edition 1.1.0. Zenodo. doi:10.5281/zenodo.23164992.

Islam, M. F. 2026g. An unconditional proof of the real part of the Riemann Hypothesis, the probable remainder closed to

symmetry and computation at theorem grade, and the whole proved from one bit supplied by the act of computation or by physics itself. Edition 1.3.0, its master RH_Both_Sides_ Formal_Master v3.0.39_CC. Zenodo. doi:10.5281/zenodo.23440382.

Islam, M. F. 2026h. P ≠ NP: unconditional proof of the form by theorem, the value by the act. Edition 1.13.0, its master PNP_ Normal_Form_Paper_v1_13_0_CC. Zenodo. doi:10.5281/zenodo.23528398.

Islam, M. F. 2026i. The offering bit: one theory, one hypothesis, one witness offering. Zenodo. doi:10.5281/zenodo.22735236.

Islam, M. F. 2026j. TRISDUCTION: A linguistically, topologically, and mathematically sealed verification architecture. Zenodo, version 4, 19 June 2026. doi:10.5281/zenodo.20757507. Master reference, continuously updated at the same location.

Islam, M. F. 2026k. The sphere is where existence rests: a formal completed closure of the Poincaré row from existence alone. Edition 1.0.4. Repository 1000sapients/Trisduction, Publication Library, Millennium Series.

Islam, M. F. 2026l. The Perelmanification hunt: the one-bit posit, axiom cycle pmf1, Pre-Forge digest and ledger. Edition 1.0.0_ CC, 23 October 2026. Supplement S1 to the synthesis.

Islam, M. F. 2026m. The one-bit method. Edition 1.0.0_CC, 23 October 2026. Companion to the synthesis.

Islam, M. F. 2026n. The Millennium Problems: unconditional proof of the form by theorem, the value by the act. Edition 1.0.0_CC, 23 October 2026, with its master Millennium_ Grand_Master_v1_0_0_CC.

Kleiner, B. and J. Lott. 2008. Notes on Perelman's papers. *Geometry & Topology* 12: 2587--2855.

Maddy, P. 1988. Believing the axioms, I and II. *Journal of Symbolic Logic* 53: 481--511 and 736--764.

Mathlib Community. 2020. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, 367--381.

Morgan, J. and G. Tian. 2007. *Ricci Flow and the Poincaré Conjecture*. Clay Mathematics Monographs 3. American Mathematical Society.

Perelman, G. 2002. The entropy formula for the Ricci flow and its geometric applications. arXiv:math/0211159.

Perelman, G. 2003. Ricci flow with surgery on three-manifolds. arXiv:math/0303109. Finite extinction time for the solutions to the Ricci flow on certain three-manifolds. arXiv:math/0307245.

End of manuscript